Building AI Agents with LangGraph — Lab Report

1. Setup

- **LLM:** qwen3:8b (Q4_K_M), served locally by Ollama on an NVIDIA GPU, through `langchain_ollama.ChatOll` Settings: `temperature=0`, `seed=42`, `reasoning=False`. There is no OpenAI key in this run. `ChatOllama` has the same interface as `ChatOpenAI`, so the lab code is unchanged. Setting `PROVIDER=openai` and `OPENAI_API_KEY` switches to `gpt-4o-mini`.
- **Versions:** langgraph 1.2.19, langchain 1.4.8, langchain-core 1.7.4, langchain-ollama 1.1.0, Python 3.11.
- Every cell of `langgraph_lab.ipynb` ran without errors. The same results came out on two runs in a row.

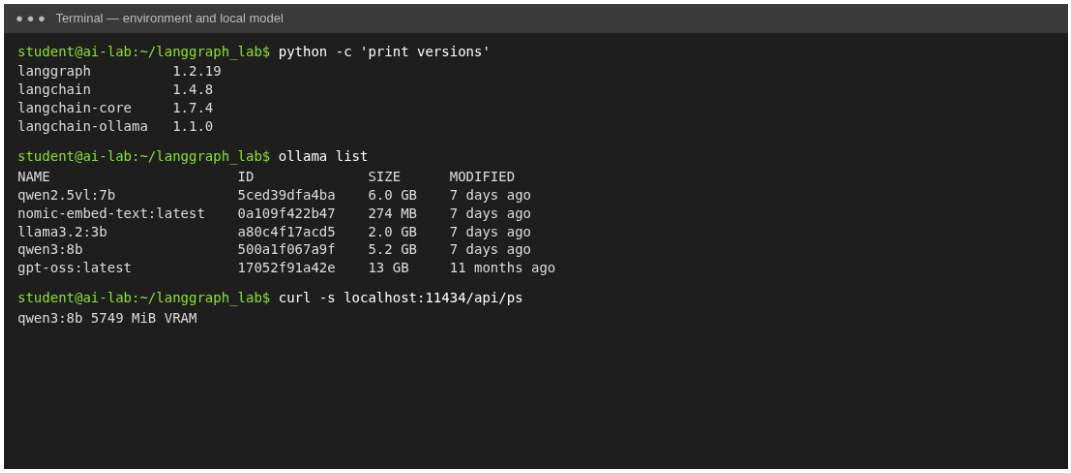


Figure 1: Environment

2. Conceptual questions

Exercise 1 — Roles of LangChain, LangGraph and the LLM

1. **LangChain** is the toolbox of building blocks for LLM applications. It gives one common interface to many models (`ChatOpenAI`, `ChatOllama`, ...), and it also provides messages, prompt templates, output parsers, retrievers and tools (`@tool`, `bind_tools`). With it you compose mostly *linear* pipelines (chains): prompt → model → parser.
2. **LangGraph** is built on LangChain. It is used for *stateful, multi-step, agentic* workflows. An application is a **graph**: a typed shared **state**, **nodes** (functions), and **edges**, including **conditional edges** and **cycles** (for example agent → tool → agent). It also adds checkpointing (memory per `thread_id`, resuming, human-in-the-loop) and streaming. LangChain says *how to call* a model or a tool; LangGraph says *what to do next* and *what to remember*.
3. **The LLM** is the reasoning and generation engine inside the nodes. It reads messages and returns either text or structured *tool calls*. It is stateless (each call only sees what is in its

prompt), so memory, routing and tool execution are handled around it by LangGraph. Here it is `qwen3:8b`, run locally by Ollama.

Exercise 2 — Nodes, edges, START, END, state

Result: `{'message': 'Hello LangGraph!', 'response': 'Agent received: Hello LangGraph!'}` (as expected).

1. A **node** is a unit of work: a Python function (or runnable) that receives the current state and returns a *partial update*, here `{"response": ...}`. Examples: a call to the LLM, a tool executor, a classifier.
2. An **edge** is a transition. It says which node runs after another one. A normal edge is fixed (`process → END`). A *conditional* edge chooses the next node at run time from the state.
3. **`START`** is the virtual entry point of the graph. The edge `START → process` says which node receives the input state first.
4. **`END`** is the virtual exit point. When the flow reaches `END`, execution stops and `invoke` returns the final state.
5. Nodes are independent functions that do not call each other. The **state** is their only shared memory and their contract. Each node reads what earlier nodes produced (for example `analysis` is used by `answer`) and returns only the keys it changes. LangGraph merges these updates (with reducers such as `add_messages`). Because the state is explicit, it can be checkpointed (memory), inspected, debugged and used for routing decisions.

3. Results of each part

Part	What was observed
III–IV LLM node, two-stage agent	The analyzer → answer pipeline gives a structured explanation of reinforcement learning (agent, environment, reward, policy, algorithms).
V–VI Router	“derivative of x^2” → mathematics (answer 2x), “REST API with Python” → programming (Flask code), “reinforcement learning” → artificial_intelligence.
VII Tools	`calculator("25 * 4 + 10")` = 110. The safe version rejects `__import__('os')`. `current_datetime` works.
VIII Tool agent	“125 * 38”: AIMessage with a tool call → ToolMessage 4750 → final answer. “Capital of Morocco”: no tool, “Rabat”.
IX Memory	Without a checkpointer the agent does not know the name. With `MemorySaver` + `thread_id` it answers “Your name is Ahmed”. A different `thread_id` does not know it.
Evaluation	Routing accuracy 6/6 = 100 %. Extended set (9 questions): free-text 9/9, structured/enum 7/9.
X Capstone	All 5 categories routed correctly. `calculator` (125·38+17 = 4767), `current_datetime` (today’s date) and `course_lookup` (AI503 on Thursdays 09:00–12:00) were called when needed. The empty question goes to the fallback. Memory works per thread. Invalid tool calls give error ToolMessages, not crashes.

4. Evaluation discussion — where did the router fail?

On the 6 lab test cases the free-text router (`qwen3:8b`, temperature 0) made **no mistake: 6/6 = 100%**. It also got 9/9 on the extended set below, which adds harder questions that mix fields, such as *"Implement gradient descent in Python"*, where maths/ML vocabulary appears in a programming request. With such a small test set this does not prove the router is robust. The weak points are in the design:

- **Fragile parsing.** The label is free text, and we only check `"mathematics" in category`. If the model answers *"Category: Math"*, adds an explanation, or writes *"AI"*, the substring test fails. The router then silently falls back to `ai`, because `route_question` sends everything unknown there. A wrong route raises no error, so it can go unnoticed.
- **Ambiguous classes.** "Gradient descent in Python" or "probability of a neural network output" fit two classes. The prompt does not say which one wins.

Ways to make it more robust: 1. **Structured output + enum validation.** `with_structured_output` with a `Literal[...]` field makes the label always valid, so no parsing is needed. *Our test (extra cell below) shows the limit of this:* the structured router produced only valid labels, but it scored **7/9**. It sent *"Explain overfitting"* and *"Why does a transformer use attention..."* to `programming`, probably because of the wording of our field description ("models, training" under AI, "data structures" under programming). A valid format does not guarantee a correct label. 2. **Few-shot examples** in the prompt, especially for borderline cases, and an explicit tie-break rule (for example "if the user asks for code, choose programming"). 3. **Validation + fallback + logging.** Map synonyms (`math` → `mathematics`), use a low-confidence/"other" class that asks the user to clarify, and log misroutes. 4. **A bigger, labelled evaluation set** (tens of questions per class, including adversarial ones), re-run after every prompt or model change. We could also add a confidence score or ask the model twice and vote (self-consistency).

5. Capstone — Intelligent University Assistant

Architecture. A `classifier` sends the question to one of five agents (COURSES, PROGRAMMING, MATHEMATICS, ADMINISTRATION, GENERAL), or to a `fallback` node for an empty question or an error. Each agent is the same LLM with a different system prompt and the tools bound to it. If the agent emits a tool call, `route_after_agent` goes to the shared `tools` node. Then `route_after_tools` goes back to the **same** agent (read from `state["category"]`), which writes the final answer. A `MemorySaver` checkpointer keeps the message history per `thread_id`.

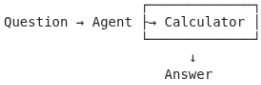
Requirements.

1. Classifier: `classify`, with 5 categories and validation of the label.
2. Specialized agents: `make_agent` factory, one system prompt per category.
3. Memory: `MemorySaver` + `thread_id`.
4. Tools: safe `calculator`, `current_datetime`, and the extension `course_lookup` (SQLite course catalogue).
5. Conditional routing: three routing functions.
6. Error handling:
   - an empty input goes to `fallback`;
   - an unknown label becomes GENERAL;
   - an API error goes to `fallback`, or a polite message from the agent;
   - an invalid tool call is caught by `handle_tool_errors=True`;
   - an invalid expression makes the calculator return "Invalid expression".

Limits observed.

- For the ADMINISTRATION questions the agent correctly refuses to invent rules. Without an institutional data source, though, it can only send the user to the office.
- The free-text classifier works with qwen3:8b but depends on the model following the instructions; this is discussed in §4.

PART VIII — Tool-using agent



The agent must **decide** whether it needs the calculator. Expected behavior for "*What is 125 * 38?*":

`Agent → recognizes math problem → calls calculator → receives result → generates answer`

We bind the tools to the LLM, so it can emit *tool calls*. A conditional edge sends the flow to the `tools` node when a tool call is present, then loops back to the agent.

In [21]:
```python
from langgraph.graph import MessagesState
from langgraph.prebuilt import ToolNode, tools_condition

tools = [calculator, current_datetime]
llm_with_tools = llm.bind_tools(tools)

def agent(state: MessagesState):
    return {"messages": [llm_with_tools.invoke(state["messages"])]}

tool_builder = StateGraph(MessagesState)
tool_builder.add_node("agent", agent)
tool_builder.add_node("tools", ToolNode(tools))

tool_builder.add_edge(START, "agent")
tool_builder.add_conditional_edges("agent", tools_condition)  # -> "tools" or END
tool_builder.add_edge("tools", "agent")                       # loop back with the result

tool_graph = tool_builder.compile()
```

In [22]:
```python
for q in ["What is 125 * 38?", "What is the capital of Morocco?"]:
    result = tool_graph.invoke({"messages": [("user", q)]})
    print("=" * 60)
    print("Q:", q)
    for m in result["messages"]:
        m.pretty_print()
```

```
============================================================
Q: What is 125 * 38?
================================ Human Message =================================

What is 125 * 38?
================================== Ai Message ==================================
Tool Calls:
  calculator (f8107d10-9100-48a0-90e7-5b6fd35f17d3)
 Call ID: f8107d10-9100-48a0-90e7-5b6fd35f17d3
  Args:
    expression: 125 * 38
================================= Tool Message =================================
Name: calculator

4750
================================== Ai Message ==================================

The result of $125 \times 38$ is **4750**.
============================================================
Q: What is the capital of Morocco?
================================ Human Message =================================

What is the capital of Morocco?
================================== Ai Message ==================================

The capital of Morocco is Rabat.
```

Figure 2: Tool-using agent trace

Evaluating the behavior of an agentic workflow

A minimal evaluation harness: define test cases with expected categories, run them, and compute accuracy. You can extend this with tool-call checks, latency, and cost.

In [27]:
```python
import time

test_cases = [
    ("What is the derivative of x²?", "mathematics"),
    ("Solve 2x + 3 = 11", "mathematics"),
    ("How do I reverse a list in Python?", "programming"),
    ("Write a function to check for palindromes.", "programming"),
    ("What is a convolutional neural network?", "artificial_intelligence"),
    ("Explain overfitting.", "artificial_intelligence"),
]

correct, rows = 0, []
for q, expected in test_cases:
    t0 = time.time()
    out = router_graph.invoke({"question": q, "category": "", "answer": ""})
    got = out["category"]
    ok = expected in got
    correct += ok
    rows.append((q, expected, got, ok, round(time.time() - t0, 2)))

import pandas as pd
df = pd.DataFrame(rows, columns=["question", "expected", "got", "correct", "seconds"])
display(df)
print(f"Routing accuracy: {correct}/{len(test_cases)} = {correct/len(test_cases):.0%}")
```

	question	expected	got	correct	seconds
0	What is the derivative of x²?	mathematics	mathematics	True	0.88
1	Solve 2x + 3 = 11	mathematics	mathematics	True	0.83
2	How do I reverse a list in Python?	programming	programming	True	2.06
3	Write a function to check for palindromes.	programming	programming	True	1.67
4	What is a convolutional neural network?	artificial_intelligence	artificial_intelligence	True	3.49
5	Explain overfitting.	artificial_intelligence	artificial_intelligence	True	4.06

```
Routing accuracy: 6/6 = 100%
```

Extra — a more robust router: structured output with an enum

We remove free-text parsing: the model must fill a Pydantic schema whose `category` field is a `Literal` (an enum). Ollama passes the JSON schema to the model as a grammar constraint, so the output can only be one of the three labels. We also add a short description of each class. Then we re-run the same evaluation, with a few harder or ambiguous questions added.

In [28]:
```python
from typing import Literal
from pydantic import BaseModel, Field

class Route(BaseModel):
    category: Literal["mathematics", "programming", "artificial_intelligence"] = Field(
        description="mathematics: calculus, algebra, equations, arithmetic. "
                    "programming: writing or debugging code, languages, APIs, data structures. "
                    "artificial_intelligence: ML/AI concepts, models, training, evaluation.")

structured_router = llm.with_structured_output(Route, method="json_schema")

def classify_structured(state: AgentState):
    try:
        return {"category": structured_router.invoke(f"Classify this question: {state['question']}").category}
    except Exception:
        return {"category": "artificial_intelligence"}     # validated fallback

b2 = StateGraph(AgentState)
b2.add_node("classifier", classify_structured)
for name, fn in [("math", mathematics_agent), ("programming", programming_agent), ("ai", ai_agent)]:
    b2.add_node(name, fn)
    b2.add_edge(name, END)
b2.add_edge(START, "classifier")
b2.add_conditional_edges("classifier", route_question, {"math": "math", "programming": "programming", "ai": "ai"})
robust_router = b2.compile()

hard_cases = test_cases + [
    ("Implement gradient descent in Python for linear regression.", "programming"),
    ("What is the probability of getting two heads in three coin tosses?", "mathematics"),
    ("Why does a transformer use attention instead of recurrence?", "artificial_intelligence"),
]

def evaluate(classifier_fn, cases):
    rows = []
    for q, expected in cases:
        t0 = time.time()
        got = classifier_fn({"question": q, "category": "", "answer": ""})["category"]
        rows.append((q, expected, got, expected in got, round(time.time() - t0, 2)))
    return pd.DataFrame(rows, columns=["question", "expected", "got", "correct", "seconds"])

# we only evaluate the classifier node (routing decision), so no answers are generated here
d_free, d_struct = evaluate(classify_question, hard_cases), evaluate(classify_structured, hard_cases)
cmp = d_free[["question", "expected"]].assign(free_text=d_free["got"], structured=d_struct["got"])
display(cmp)
print(f"Free-text router : {d_free.correct.sum()}/{len(hard_cases)}   avg {d_free.seconds.mean():.2f}s")
print(f"Structured router: {d_struct.correct.sum()}/{len(hard_cases)}   avg {d_struct.seconds.mean():.2f}s")
```

	question	expected	free_text	structured
0	What is the derivative of x²?	mathematics	mathematics	mathematics
1	Solve 2x + 3 = 11	mathematics	mathematics	mathematics
2	How do I reverse a list in Python?	programming	programming	programming
3	Write a function to check for palindromes.	programming	programming	programming
4	What is a convolutional neural network?	artificial_intelligence	artificial_intelligence	artificial_intelligence

Figure 3: Evaluation

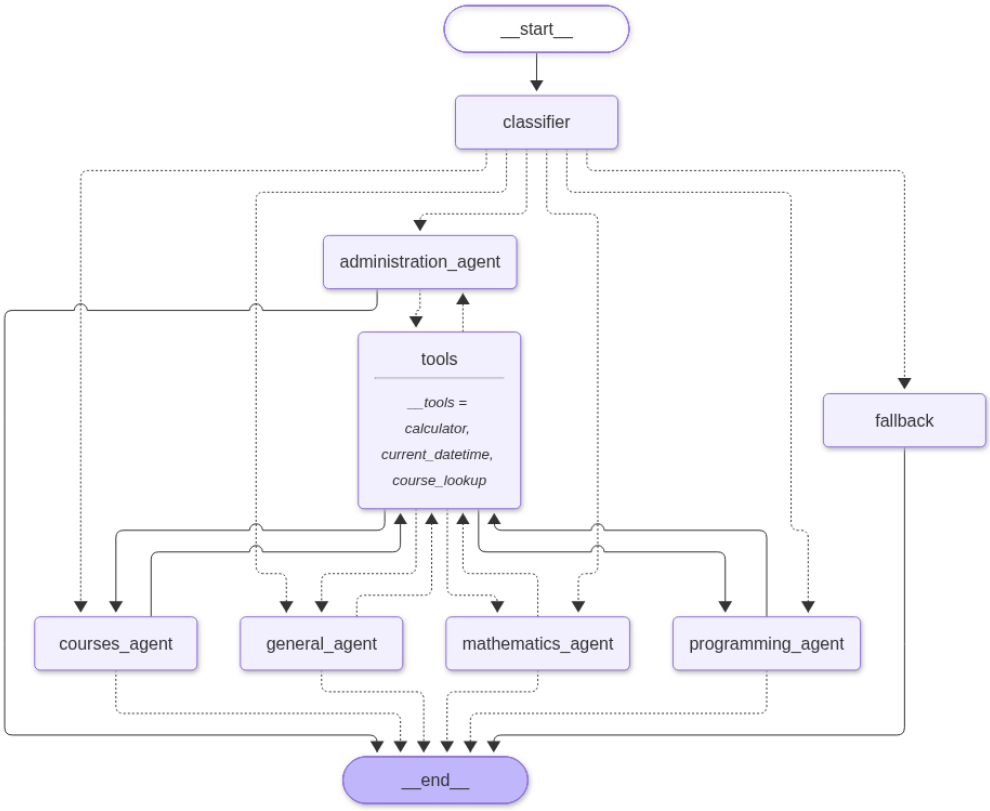


Figure 4: Architecture of the assistant

- Each question costs two LLM calls (classification + answer), and three if a tool is used.

6. Going further: the Campus Copilot web app

The capstone was extended into a full application (`app/`, documented in `docs/ARCHITECTURE.md`):

- **Structured, conversation-aware routing.** A Pydantic `Route` schema (enum + confidence + reason) is passed to Ollama as a JSON-schema constraint, with few-shot examples and the last 4 turns. Follow-up questions ("which course fits *that* interest?") are routed correctly.
- **MCP tools.** The course catalogue, academic calendar and administrative procedures are served by a separate **Model Context Protocol** server (FastMCP, stdio) and discovered with `langchain-mcp-adapters`.
- **Least privilege.** Each specialist agent is bound only to the tools of its domain.
- **Durable memory.** `AsyncSqliteSaver` writes one checkpoint per step, so conversations survive restarts. `trim_messages` bounds the context sent to the model.
- **Reliability.** `RetryPolicy` (3 attempts, exponential back-off) on LLM nodes; tool errors become ToolMessages; the empty input goes to the fallback without calling the LLM.
- **Streaming UI.** FastAPI + Server-Sent Events stream the tokens, node updates and tool calls. A run inspector shows the route, the graph path with per-node latency, the tool calls and the state.
- **Hand-drawn architecture** (rough.js), a docs page and an evaluation page.
- **Quality.** 11 pytest tests pass. The evaluation on 16 labelled questions gives 100 % routing accuracy, 100 % correct tool selection, 100 % grounded answers and about 1.1 s average latency.



Figure 5: Capstone demo

```
● ● ●  Terminal — interactive chat with memory and tools

student@ai-lab:~/langgraph_lab$ printf '<3 questions + exit>' | python src/assistant.py --chat
(stdin)
My name is Sara and I study computer vision.
What is my name and what do I study?
How much is 2**10 - 24?
exit

University Assistant (type 'exit' to quit)
==================================================================
USER     : My name is Sara and I study computer vision.
CATEGORY : GENERAL | tools: none
ASSISTANT: Nice to meet you, Sara! I'm here to help with any questions you might have about your studies, courses, or anything else. How
can I assist you today?
==================================================================
USER     : What is my name and what do I study?
CATEGORY : GENERAL | tools: none
ASSISTANT: Your name is Sara, and you study computer vision. Let me know if you need help with anything related to your studies!
==================================================================
USER     : How much is 2**10 - 24?
CATEGORY : MATHEMATICS | tools: calculator
ASSISTANT: The result of $ 2^{10} - 24 $ is **1000**. Let me know if you need further assistance!
```

Figure 6: Interactive chat with memory and tools

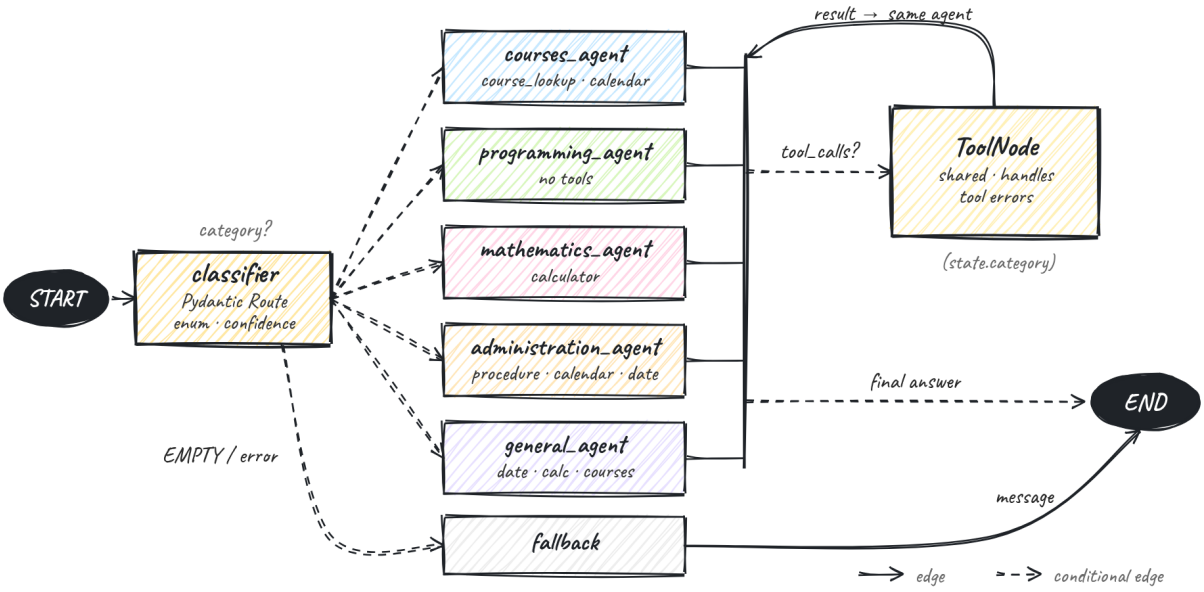


Figure 7: Agent graph of the web app

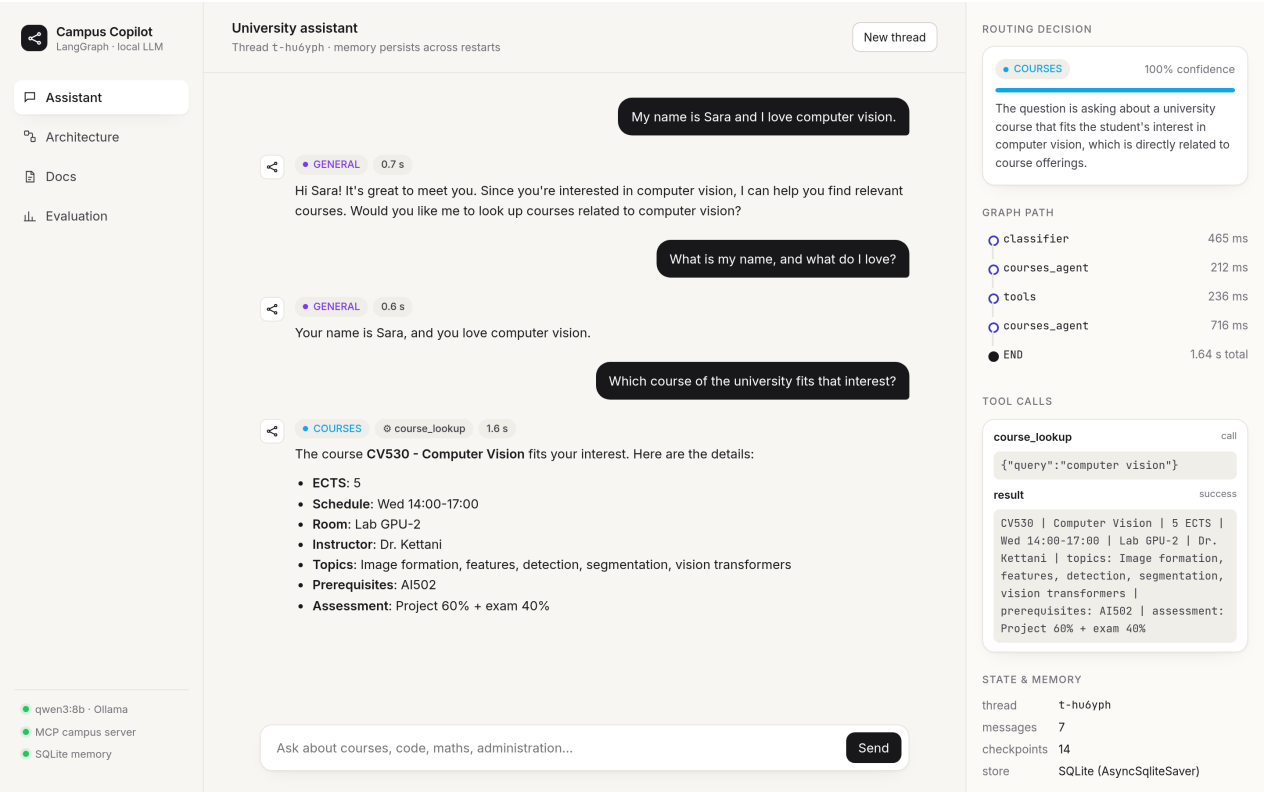


Figure 8: Web app: memory and context-aware routing

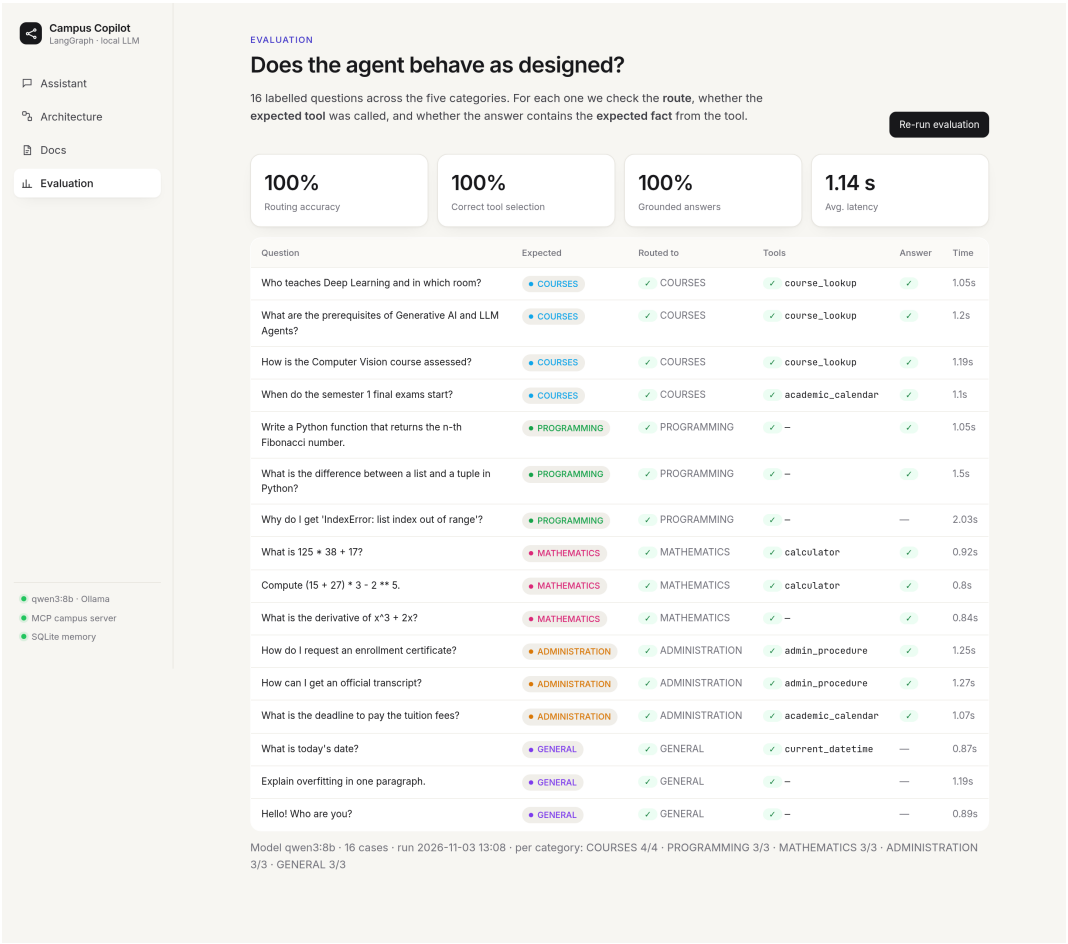


Figure 9: Web app: evaluation